# Vulnerability Assessment Report

## Imagur Vulnerability Assessment Report

**Imagur**
132 Morphett Rd
Glengowrie SA 5044, Australia

**Flinders University**
Sturt Road, Bedford Park SA 5042

October 4, 2026

Report Version: 1.0

*CONFIDENTIAL*

# Table of Contents

List of Figures ..... 4

1 Document Control ..... 5

1.1 Team ..... 5

1.2 List of Changes ..... 5

2 Executive Summary ..... 6

2.1 Overview ..... 6

2.2 Identified Vulnerabilities ..... 7

3 Methodology ..... 9

3.1 Objective ..... 9

3.2 Scope ..... 9

3.3 User Accounts and Permissions ..... 10

4 Findings ..... 11

C1: Hard-coded AWS credential material in public JavaScript ..... 11

H1: Stored XSS in image filename on the image detail page ..... 12

H2: IDOR — upload form trusts client-supplied user_id ..... 13

M1: Session cookie missing Secure, HttpOnly, and SameSite ..... 14

M2: HTTPS directory URLs redirect to HTTP ..... 15

M3: Unauthenticated image-existence oracle on the JSON API ..... 16

M4: Missing HSTS ..... 17

M5: SQL error messages disclose database internals ..... 18

I1: Missing Content-Security-Policy ..... 19

I2: PHP version disclosed ..... 20

I3: Registration email check is not enforced ..... 21

5 Disclaimer ..... 22

A Appendix ..... 23

A.1 Static Appendix Section ..... 23

**A.2 Methodology ..... 23**
**A.3 Limitations ..... 23**
**A.4 Coverage ..... 23**

# List of Figures

Figure 1 - Distribution of identified vulnerabilities ..... 8

# 1 Document Control

## 1.1 Team

| Contact | Details | Role |
|---|---|---|
| ClawSec | Autonomous agent | Pentester |

## 1.2 List of Changes

| Version | Description | Date |
|---|---|---|
| 1.0 | Final report | Oct 4, 2026 |

# 2 Executive Summary

## 2.1 Overview

# Executive Summary

**Imagur** operates **Imager**, a public image-sharing website at https://imagesharer.xyz/. People can register, sign in, and upload private or public images. The site is hosted as PHP 8.2.34 on nginx.

This report covers an authorized security assessment of the main website only. Permission was confirmed on that site before work began. Testing ran on 4 October 2026.

## Overall result

The assessment confirmed **11 findings**. The verified findings file rated them as **3 high**, **3 medium**, **3 low**, and **2 informational**. CVSS v3.1 vectors were copied unchanged from that file. In this report those vectors score as **1 critical**, **2 high**, **5 medium**, and **3 informational**: the public-script cloud-key vector scores 10.0; missing HSTS and the image-existence oracle score as medium; missing Content-Security-Policy has no vector and is shown as informational.

The most urgent issues are:

1. **Cloud login keys in a public script.** Anyone can download a JavaScript file from the website that contains cloud access-key material. That file should be removed at once, and every key that has ever been in it should be replaced. The keys were not used against the cloud during this assessment, so live access was not confirmed.
2. **Image filenames that can run script in the browser.** A signed-in user can upload an image whose filename includes HTML. When that image is opened, the browser can run the attacker's script and steal the user's login, because the session cookie is readable by script.
3. **Uploads into another user's private library.** The upload form trusts a hidden user ID from the browser. A signed-in user who knows another user's ID can place files into that user's private library.

Also fix the session cookie flags, stop HTTPS pages from redirecting to HTTP, and stop showing raw database errors to users.

## What this means for the business

- Cloud keys in a public file are a high-priority incident-response item even if they later prove unused. Treat them as exposed until they are rotated.
- A visitor who is signed in can be tricked into running attacker script on the image page, which can take over that session.
- One customer can modify another customer's private library without consent.
- Weak cookie flags and HTTPS-to-HTTP redirects increase the chance that a login cookie is sent in the clear.

## Recommended priority order

1. Remove `/js/awsvalid.js` from the website and rotate every AWS access key that has ever been in that file.
2. Encode image filenames before showing them, and set the session cookie to HttpOnly (and Secure / SameSite).
3. Ignore client-supplied `user_id` on upload and bind files to the signed-in account on the server.
4. Return generic errors instead of raw database messages; keep HTTPS redirects on HTTPS; enable HSTS.
5. Address the remaining low and informational items in the next maintenance cycle (image-ID oracle, Content-Security-Policy, PHP version header, and the unused email-verification hint).

We did not test extra server ports or the file-storage cloud account, because those were left out of scope by the owner.

## 2.2 Identified Vulnerabilities

| # | CVSS | Description | Page |
|---|---|---|---|
| C1 | 10.0 | Hard-coded AWS credential material in public JavaScript | 11 |
| H1 | 7.6 | Stored XSS in image filename on the image detail page | 12 |
| H2 | 7.1 | IDOR — upload form trusts client-supplied user_id | 13 |
| M1 | 6.5 | Session cookie missing Secure, HttpOnly, and SameSite | 14 |
| M2 | 5.3 | HTTPS directory URLs redirect to HTTP | 15 |
| M3 | 5.3 | Unauthenticated image-existence oracle on the JSON API | 16 |
| M4 | 4.8 | Missing HSTS | 17 |
| M5 | 4.3 | SQL error messages disclose database internals | 18 |
| I1 | 0.0 | Missing Content-Security-Policy | 19 |
| I2 | 0.0 | PHP version disclosed | 20 |
| I3 | 0.0 | Registration email check is not enforced | 21 |

### Vulnerability Overview

In the course of this penetration test **1 Critical**, **2 High**, **5 Medium** and **3 Info** vulnerabilities were identified:

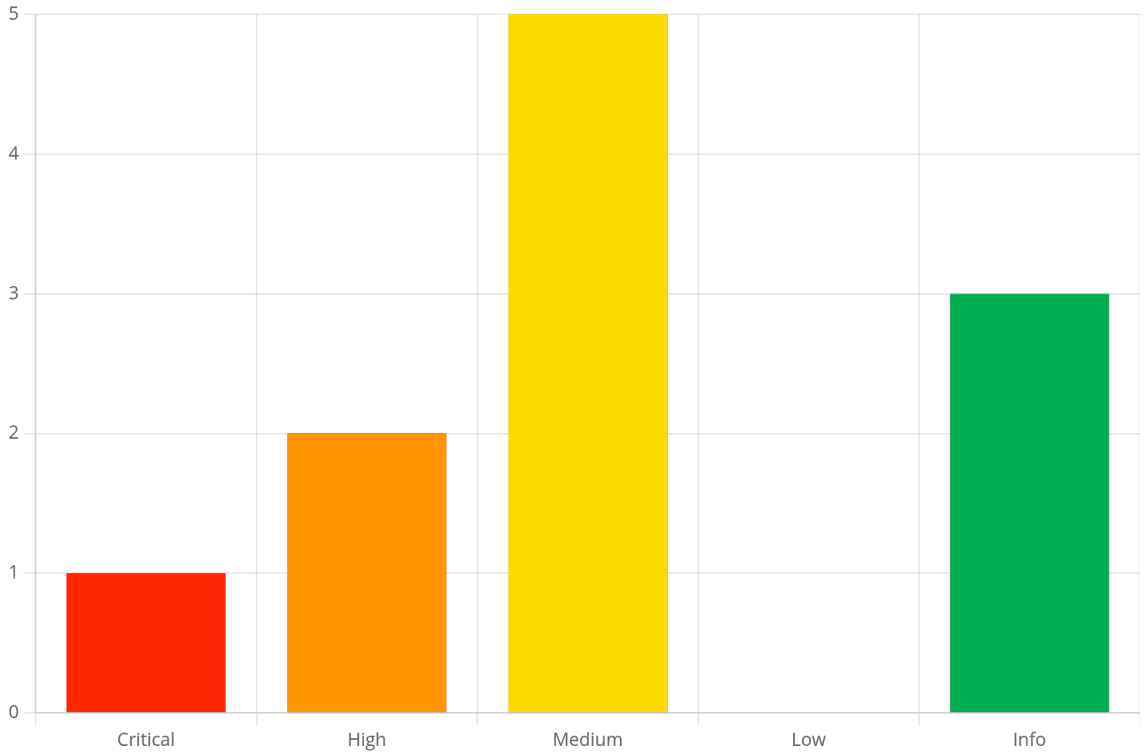



**Figure 1 - Distribution of identified vulnerabilities**

# 3 Methodology

This entire vulnerability assessment process was completed using agentic AI and existing LLM models. The purpose of such assessments is to automatically

## 3.1 Objective

The entire objective of this research is to evaluate the effectiveness of autonomously evaluate security posture for any business available on Google Map without the needs of technical guidance to determine how feasiable, seamless and cost-affective such approach is.

## 3.2 Scope

This assessment covered only the owner-approved main website. Authorization was confirmed on that site on 4 October 2026 before testing. Active testing ran from 22:40 UTC to 23:05 UTC the same day.

**In scope (owner approved)**

| System | Description |
|---|---|
| https://imagesharer.xyz/ | Main Imager website (apex host and application paths only) |
| imagesharer.xyz | Apex domain of the approved website |

**Enumerated but owner-excluded (inventory only; not tested)**

- www.imagesharer.xyz
- resources.imagesharer.xyz
- CloudFront d273uevi0662l0.cloudfront.net and likely S3 origin
- Origin IP 51.161.131.44 extra ports, including TCP 5432
- cataierc.com (shared IP; owner said not theirs)
- OVH netblock remainder, Spaceship DNS, Flinders University email infrastructure
- Lookalike domains (including imager.xyz, imagur.xyz, fshare.xyz, and others)
- FShare Google Maps listing

**Enumerated related assets (not in the approved test set)**

- confirmed org inventory: imagesharer.xyz, www.imagesharer.xyz, resources.imagesharer.xyz, 51.161.131.44
- cloud inventory: CloudFront distribution for resources, likely S3 origin, public JavaScript indicating AWS IAM
- rejected: cataierc.com, imager.xyz, imagur.xyz, fshare.xyz, other lookalikes

**Allowed methods**

Unauthenticated and single test-account web testing, directory discovery, non-destructive injection, and ReDoS.

**Forbidden methods**

Brute force, credential stuffing, password spraying, DDoS, malware, phishing, MFA bypass, lateral movement, destructive actions, and using leaked cloud keys against AWS.

## 3.3 User Accounts and Permissions

No owner-supplied staff accounts were used. Authenticated tests used two tester-created accounts only. Account passwords are not included in this report.

# 4 Findings

| C1: Hard-coded AWS credential material in public JavaScript | |
|---|---|
| Score | 10.0 (Critical) |
| Vector string | CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:C/C:H/I:H/A:N |
| Target | https://imagesharer.xyz/js/awsvalid.js |
| References | • CWE-798<br>• CWE-200 |

## Overview

A publicly downloadable script on the website contains AWS access-key-style material and code that calls AWS STS GetCallerIdentity.

## Details

GET /js/awsvalid.js returns live JavaScript (about 5298 bytes). It includes GetCallerIdentity against sts.amazonaws.com, region strings ap-southeast-2 and us-east-1, one AKIA-format access key on a non-comment line, and secret-key-like fields. The login layout currently comments out the script tag, but the file itself is world-readable with no authentication. The keys were not used against AWS during this assessment, so live privilege was not confirmed. If they are valid, anyone on the internet can impersonate the associated cloud identity. Secret values are not included in this report.

## Recommendation

Remove the file from the web root immediately. Rotate every AWS access key that has ever been in this file. Store cloud credentials only in server-side secrets, never in client JavaScript. Scan the repository and deployment bundle for other leaked keys.

| H1: Stored XSS in image filename on the image detail page | |
|---|---|
| Score | 7.6 (High) |
| Vector string | CVSS:3.1/AV:N/AC:L/PR:L/UI:R/S:C/C:H/I:L/A:N |
| Target | https://imagesharer.xyz/images/{id} |
| References | CWE-79 |

## Overview

A logged-in user can upload an image whose filename contains HTML. When that image is opened, the filename is inserted into the page without encoding and the browser runs the attacker's script.

## Details

The upload form at POST /images/upload stores the original filename and later renders it in the image detail view inside an unescaped HTML span. A test file named `<img src=x onerror=alert(1)>.png` produced this markup on the detail page: a raw img tag with an onerror handler. Title, heading, and alt text were encoded, but the "Filename" row was not. The session cookie PHPSESSID is not marked HttpOnly, so this script can read the session. Combined with the user_id upload flaw, an attacker can also plant such a file in another user's library so that the victim triggers the script while viewing their own image.

## Recommendation

HTML-encode filenames everywhere they are shown. Reject or sanitise characters such as < > " ' in filenames. Set HttpOnly on the session cookie. Add a Content-Security-Policy that disallows inline script.

| H2: IDOR — upload form trusts client-supplied user_id | |
|---|---|
| Score | 7.1 (High) |
| Vector string | CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:L/I:H/A:N |
| Target | https://imagesharer.xyz/images/upload |
| References | • CWE-639<br>• CWE-915 |

## Overview

The upload page includes a hidden user_id field. The server uses that value when saving the image, so a logged-in user can place files into another user's private library if they know that user's ID.

## Details

GET /images/upload returns a hidden input user_id set to the current account UUID. POST /images/upload inserts that value into the images table (confirmed by a PostgreSQL foreign-key error when a fake UUID was sent). Using a second test account's UUID while authenticated as the first account stored a test file in the second account's dashboard under that user's storage prefix. Image detail pages for other users' private files were not readable by the uploader, but the victim's library was modified without their consent. User IDs are shown on /profile, so they are available to anyone who can view or phish a profile, and they are also present in upload HTML.

## Recommendation

Ignore client-supplied user_id. Bind uploads to the authenticated session user on the server. Treat user IDs as internal, not as a writable form field.

| M1: Session cookie missing Secure, HttpOnly, and SameSite | |
|---|---|
| Score | 6.5 (Medium) |
| Vector string | CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:L/A:N |
| Target | https://imagesharer.xyz/ |
| References | • CWE-614<br>• CWE-1004<br>• CWE-1275 |

## Overview

The PHP session cookie is not marked Secure, HttpOnly, or SameSite, so it can be read by script and may be sent over HTTP.

## Details

Set-Cookie for PHPSESSID is issued with only path=/. Other cookies (page_views, last_login) correctly use Secure, HttpOnly, and SameSite=Lax. Because HttpOnly is missing, the stored XSS finding can steal the session. Because Secure is missing, the cookie can be sent on unencrypted HTTP, including after the HTTPS-to-HTTP redirects described separately. last_login also stores a copy of the session id inside its value. Session identifier values are not included in this report.

## Recommendation

Set PHPSESSID with Secure; HttpOnly; SameSite=Lax (or Strict). Do not duplicate the session id in other cookies. Prefer session.cookie_secure, session.cookie_httponly, and session.cookie_samesite in PHP.

| M2: HTTPS directory URLs redirect to HTTP | |
|---|---|
| Score | 5.3 (Medium) |
| Vector string | CVSS:3.1/AV:N/AC:H/PR:N/UI:R/S:U/C:H/I:N/A:N |
| Target | • https://imagesharer.xyz/uploads<br>• https://imagesharer.xyz/js<br>• https://imagesharer.xyz/css |
| References | • CWE-319<br>• CWE-757 |

## Overview

Some HTTPS links are redirected to http:// on the same host, which can expose the session cookie on the clear-text hop.

## Details

Requests to https://imagesharer.xyz/uploads, /js, and /css (no trailing slash) return 301 Location: http://imagesharer.xyz/.../ (http scheme). A following HTTP request is then redirected back to HTTPS. Combined with PHPSESSID lacking the Secure flag, the browser may attach the session cookie to that HTTP request. Directory listings themselves returned 403.

## Recommendation

Issue slash-normalising redirects only to https://. Enable HSTS so browsers refuse the HTTP hop.

| M3: Unauthenticated image-existence oracle on the JSON API | |
|---|---|
| Score | 5.3 (Medium) |
| Vector string | CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N |
| Target | https://imagesharer.xyz/api/v1/images/{id} |
| References | CWE-203 |

## Overview

The public API answers differently for real image IDs versus unknown IDs, so anyone can test whether an image ID exists.

## Details

GET /api/v1/images/{existing-uuid} without a token returns 403 FORBIDDEN “You do not have access to this image.” GET /api/v1/images/{random-uuid} returns 404 “Image not found.” Invalid non-UUID paths return a different 404. /api/v1/health is unauthenticated and returns {"status":"ok","service":"imager"}. /api/v1/images without an ID returns 401 asking for an authentication token.

## Recommendation

Require authentication first. Return the same 404 for missing and forbidden objects. Do not leak service identity on health if that is not intended to be public.

| M4: Missing HSTS | |
|---|---|
| Score | 4.8 (Medium) |
| Vector string | CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:L/I:L/A:N |
| Target | https://imagesharer.xyz/ |
| References | CWE-319 |

## Overview

The site does not send Strict-Transport-Security, so browsers are not told to stay on HTTPS.

## Details

Response headers on the login and dashboard flows include X-Content-Type-Options, X-Frame-Options, and Referrer-Policy, but not Strict-Transport-Security. That leaves room for SSL stripping and for the HTTP downgrade redirects above.

## Recommendation

Send Strict-Transport-Security: max-age=31536000; includeSubDomains on all HTTPS responses, after confirming HTTP-to-HTTPS works for every path.

| M5: SQL error messages disclose database internals | |
|---|---|
| Score | 4.3 (Medium) |
| Vector string | CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:L/I:N/A:N |
| Target | https://imagesharer.xyz/images/upload |
| References | CWE-209 |

## Overview

Invalid upload input returns a 500 JSON body that includes raw PostgreSQL error text, table names, and constraint names.

## Details

POST /images/upload with a user_id that does not exist returned HTTP 500 JSON INTERNAL_ERROR whose detail field contained SQLSTATE[23503], the images table, and constraint images_user_id_fkey, including the submitted key. That confirms PostgreSQL, the schema around images.user_id, and that the client field is used in an INSERT. It helps an attacker refine further attacks.

## Recommendation

Return a generic error to users. Log full SQL errors only on the server. Disable detailed exception output in production.

| I1: Missing Content-Security-Policy | |
|---|---|
| Score | 0.0 (Info) |
| Vector string | N/A |
| Target | https://imagesharer.xyz/ |
| References | CWE-693 |

## Overview

Pages do not send a Content-Security-Policy header, so a successful XSS has no extra browser brake.

## Details

Security headers include nosniff, DENY framing, and Referrer-Policy, but no CSP. That makes the stored XSS finding easier to exploit.

## Recommendation

Add a strict CSP (default-src 'self'; object-src 'none'; base-uri 'self') and avoid unsafe-inline if possible.

| I2: PHP version disclosed | |
|---|---|
| Score | 0.0 (Info) |
| Vector string | N/A |
| Target | https://imagesharer.xyz/ |
| References | CWE-200 |

## Overview

Every PHP response includes X-Powered-By: PHP/8.2.34, which tells attackers the exact runtime version.

## Details

nginx plus X-Powered-By PHP/8.2.34 appeared on login, register, dashboard, and upload responses. Version details help someone match known PHP issues.

## Recommendation

Disable expose_php and remove X-Powered-By in nginx/PHP.

| I3: Registration email check is not enforced | |
|---|---|
| Score | 0.0 (Info) |
| Vector string | N/A |
| Target | https://imagesharer.xyz/register |
| References | - |

## Overview

The sign-up form says email verification is required, but a new account reached the dashboard immediately with no verification step.

## Details

GET /register shows an email field hint that verification is required. POST /register for a test mailbox returned 302 /dashboard and a logged-in session. That is weaker than the advertised control and allows automated account creation (no password guessing was performed).

## Recommendation

Enforce verification before session issue, or remove the misleading hint. Add CSRF tokens to register, login, upload, visibility, delete, and logout forms.

# 5 Disclaimer

The purpose of this research project is to help evaluating the seamlessness, effectiveness and cost-benefit ratio of autonomously performing vulnerabilty assessment using agentic AI. I do not take any responsibility upon any misuse or abuse of such tool where it can face significant legal actions.

# A Appendix

## A.1 Static Appendix Section

## A.2 Methodology

The assessment followed a phased, non-destructive methodology:

1. Authorization check on the main public-facing website before any testing.
2. OSINT and HTTP enumeration of company-related assets.
3. Owner confirmation of in-scope versus excluded assets.
4. Mapping of login, register, dashboard, profile, upload, image detail, and logout on https://imagesharer.xyz/.
5. Review of JSON API endpoints `/api/v1/health` and `/api/v1/images`.
6. Authenticated testing with two tester-created accounts only.
7. Targeted, non-destructive checks (directory discovery, injection, ReDoS).

No brute force, credential stuffing, password spraying, denial-of-service, malware, phishing, MFA bypass, lateral movement, or destructive changes were used. Leaked cloud keys were not invoked against AWS.

## A.3 Limitations

The following limitations apply to this assessment:

- Certificate Transparency was unavailable, so extra subdomains may exist.
- Direct testing of S3/CloudFront and the database port was out of scope.
- AWS keys found in public JavaScript were not invoked.
- No brute-force of logins was performed.
- Testing was performed from a single vantage point on 4 October 2026 (22:40–23:05 UTC).
- Third-party infrastructure (OVH remainder, Spaceship DNS, Flinders University email, lookalike domains, FShare Google Maps listing) was excluded.

## A.4 Coverage

Coverage on the approved website included:

- Login, register, dashboard, profile, upload, image detail, and logout
- JSON API `/api/v1/health` and `/api/v1/images`
- Confirmed findings recorded in findings.txt: 11 total (3 high, 3 medium, 3 low, 2 informational)

The following were mapped as inventory only and were not actively tested, by owner exclusion: www.imagesharer.xyz, resources.imagesharer.xyz, CloudFront, origin extra ports including TCP 5432, cataierc.com, OVH netblock remainder, Spaceship DNS, Flinders University email infrastructure, lookalike domains, and the FShare Google Maps listing.